Dynamic Queue in C++

Generated by Doxygen 1.16.1

1 Dynamic Queue in C++ 1
1.1 Characteristics . . . 1
1.2 Project's structure . . . 1
1.3 Documentation . . . 2
1.4 Execution . . . 2
1.5 Author . . . 2
1.6 Teacher . . . 2
1.7 Subject and Academic program . . . 2
1.8 Date . . . 2

2 Tests 3
2.1 Test results . . . 3

3 Directory Hierarchy 5
3.1 Directories . . . 5

4 Class Index 7
4.1 Class List . . . 7

5 File Index 9
5.1 File List . . . 9

6 Directory Documentation 11
6.1 headers Directory Reference . . . 11
6.2 src Directory Reference . . . 11

7 Class Documentation 13
7.1 Node< T > Class Template Reference . . . 13
7.1.1 Detailed Description . . . 13
7.1.2 Constructor & Destructor Documentation . . . 14
7.1.2.1 Node() [1/3] . . . 14
7.1.2.2 Node() [2/3] . . . 14
7.1.2.3 Node() [3/3] . . . 14
7.1.3 Member Function Documentation . . . 15
7.1.3.1 getNext() . . . 15
7.1.3.2 getValue() . . . 15
7.1.3.3 insertNext() . . . 15
7.1.3.4 insertValue() . . . 16
7.2 Queue< T > Class Template Reference . . . 16
7.2.1 Detailed Description . . . 16
7.2.2 Constructor & Destructor Documentation . . . 17
7.2.2.1 Queue() . . . 17
7.2.2.2 ~Queue() . . . 17
7.2.3 Member Function Documentation . . . 17

7.2.3.1 dequeue() . . . 17
7.2.3.2 display() . . . 18
7.2.3.3 emptyQueue() . . . 18
7.2.3.4 enqueue() . . . 19
7.2.3.5 getSize() . . . 19
7.2.3.6 peek() . . . 20

8 File Documentation **21**
8.1 headers/node.h File Reference . . . 21
8.1.1 Detailed Description . . . 21
8.2 node.h . . . 21
8.3 headers/queue.h File Reference . . . 22
8.3.1 Detailed Description . . . 22
8.4 queue.h . . . 22
8.5 README.md File Reference . . . 23
8.6 src/main.cpp File Reference . . . 23
8.6.1 Detailed Description . . . 23
8.6.2 Function Documentation . . . 23
8.6.2.1 main() . . . 23
8.6.2.2 menu() . . . 24
8.7 main.cpp . . . 25
8.8 src/node.cpp File Reference . . . 26
8.8.1 Detailed Description . . . 26
8.9 node.cpp . . . 26
8.10 src/queue.cpp File Reference . . . 27
8.10.1 Detailed Description . . . 27
8.11 queue.cpp . . . 27

Index **31**

Chapter 1

Dynamic Queue in C++

Implementation of a queue in C++ using dynamic memory.

1.1 Characteristics

- Enqueue.
- Dequeue.
- Peek.
- Display.
- Automatic Doxygen documentation.

1.2 Project's structure

- build/
  - Unix executable file (exec)
- doc/
  - html/
  - latex/
- src/
  - queue.cpp
  - node.cpp
  - main.cpp
- headers/
  - queue.h
  - node.h

1.3 Documentation

All the documentation was generated using Doxygen.

To view it: Go to doc/html/index.html and open with your default browser. To check all comments: Click on "Class Members" and then on "All".

Or check the PDF file "refman.pdf".

1.4 Execution

- To run directly: Open and run the Unix executable file (exec).
- By using commands: Open your terminal, and run...

g++ src/main.cpp -Iheaders -o dynamQueue ./dynamQueue

1.5 Author

Camila Andrea Jaimes Martinez.

1.6 Teacher

Manuel Balderas Victoria.

1.7 Subject and Academic program

- Data structure.
- Software Engineering and Data Mining (1st year).

1.8 Date

March 15, 2026.

Chapter 2

Tests

Execution of the Queue program.

2.1 Test results

The following images show the execution of the Queue program in the terminal, including the functions of Enqueue, Dequeue, Peek and Display.

```
Last login: Sun Mar 15 13:56:24 on ttys000
/Users/camila/Desktop/dynam_queue/build/dynamQueue ; exit;
camila@MacBook-Air-de-Camila ~ % /Users/camila/Desktop/dynam_queue/build/dynamQueue ; exit;


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 9

The node has been added with the value of 9!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 11

The node has been added with the value of 11!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 1

What value is to be stored in the new Node?: 7

The node has been added with the value of 7!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option:
1

What value is to be stored in the new Node?: 4

The node has been added with the value of 4!
```

Figure 2.1 Queue execution! (Part 1)

```
        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 4
9|11|7|4|
The number of Nodes in the queue is: 4
--> First value of the queue (head): 9
--> Last value of the queue (tail): 4


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 2

The value stored in the front of the queue is: 9
The front of the queue has been deleted!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 4
11|7|4|
The number of Nodes in the queue is: 3
--> First value of the queue (head): 11
--> Last value of the queue (tail): 4


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 3

The value stored in the front of the queue is: 11


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 5
```

Figure 2.2 Queue execution! (Part 2)

```
--> Choose an option: 4
9|11|7|4|
The number of Nodes in the queue is: 4
--> First value of the queue (head): 9
--> Last value of the queue (tail): 4


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 2

The value stored in the front of the queue is: 9
The front of the queue has been deleted!


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 4
11|7|4|
The number of Nodes in the queue is: 3
--> First value of the queue (head): 11
--> Last value of the queue (tail): 4


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 3

The value stored in the front of the queue is: 11


        --- MENU! ---
1) Push.
2) Pop.
3) Peek.
4) Display.
5) Exit.

--> Choose an option: 5

Exiting the program...
Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Proceso completado]
```

Figure 2.3 Queue execution! (Part 3)

Chapter 3

Directory Hierarchy

3.1 Directories

headers 11
- node.h 21
- queue.h 22

src 11
- main.cpp 23
- node.cpp 26
- queue.cpp 27

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Node< T > 13
Queue< T > 16

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

headers/node.h
Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next" . 21

headers/queue.h
Defines the structure of the Queue. Its structure consists of two pointers called "head" and "tail", respectively 22

src/main.cpp
The complete program of the dynamic queue. Includes a menu function so that the user may input any desired value 23

src/node.cpp
Implementation of the constructors of the Node 26

src/queue.cpp
Implementation of the constructors of the queue 27

Chapter 6

Directory Documentation

6.1 headers Directory Reference

Files

- file node.h

  Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".
- file queue.h

  Defines the structure of the Queue. Its structure consists of two pointers called "head" and "tail", respectively.

6.2 src Directory Reference

Files

- file main.cpp

  The complete program of the dynamic queue. Includes a menu function so that the user may input any desired value.
- file node.cpp

  Implementation of the constructors of the Node.
- file queue.cpp

  Implementation of the constructors of the queue.

Chapter 7

Class Documentation

7.1 Node< T > Class Template Reference

```
#include <node.h>
```

Public Member Functions

- Node (T data, Node< T > *nextNode)

  Initializes the Node with a value and sets the pointer to the next Node.
- Node (T data)

  Initializes the Node with a value and sets its pointer to nullptr.
- Node ()

  Initializes the Node empty (no value nor reference stored).
- void insertValue (T data)

  Inserts a value into the Node.
- void insertNext (Node< T > *nextNode)

  Sets the pointer to the next Node.
- T getValue () const

  Returns the value stored in the Node.
- Node< T > * getNext () const

  Returns the memory address to the next Node.

7.1.1 Detailed Description

template<typename T>
class Node< T >

Definition at line 15 of file node.h.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 Node() [1/3]

```
template<typename T>
Node< T >::Node (
            T data,
            Node< T > * nextNode)
```

Initializes the Node with a value and sets the pointer to the next Node.

Parameters

data	The value to be stored in the Node.
nextNode	The memory address of the next Node.

Definition at line 18 of file node.cpp.

```
00019 {
00020     value = data;
00021     next = nextNode;
00022 }
```

7.1.2.2 Node() [2/3]

```
template<typename T>
Node< T >::Node (
            T data)
```

Initializes the Node with a value and sets its pointer to nullptr.

Parameters

data	The value to be stored in the Node.

Definition at line 33 of file node.cpp.

```
00034 {
00035     value = data;
00036     next = nullptr;
00037 }
```

7.1.2.3 Node() [3/3]

```
template<typename T>
Node< T >::Node ()
```

Initializes the Node empty (no value nor reference stored).

Initializes the Node empty (no value nor reference stored). Default Node constructor.

Definition at line 47 of file node.cpp.

```
00048 {
00049     value = T();
00050     next = nullptr;
00051 }
```

7.1.3 Member Function Documentation

7.1.3.1 getNext()

```
template<typename T>
Node< T > * Node< T >::getNext () const
```

Returns the memory address to the next Node.

Returns

Memory address of the next Node.

Definition at line 104 of file node.cpp.

```
00105 {
00106     return next;
00107 }
```

7.1.3.2 getValue()

```
template<typename T>
T Node< T >::getValue () const
```

Returns the value stored in the Node.

Returns

The value stored in the Node.

Definition at line 90 of file node.cpp.

```
00091 {
00092     return value;
00093 }
```

7.1.3.3 insertNext()

```
template<typename T>
void Node< T >::insertNext (
            Node< T > * nextNode)
```

Sets the pointer to the next Node.

Parameters

nextNode	The memory address of the next Node.

Definition at line 76 of file node.cpp.

```
00077 {
00078     next = nextNode;
00079 }
```

7.1.3.4 insertValue()

```
template<typename T>
void Node< T >::insertValue (
            T data)
```

Inserts a value into the Node.

Parameters

data	The value to be stored.

Definition at line 62 of file node.cpp.

```
00063 {
00064     value = data;
00065 }
```

The documentation for this class was generated from the following files:

- headers/node.h
- src/node.cpp

7.2 Queue< T > Class Template Reference

```
#include <queue.h>
```

Public Member Functions

- Queue ()

  Initializes an empty queue.
- ~Queue ()

  Clears the queue and releases all dynamically allocated resources.
- bool emptyQueue () const

  Returns true if the queue has no Nodes, and false otherwise.
- int getSize () const

  Returns the number of Nodes in the queue by traversing the list.
- void enqueue (Node< T > *data)

  Adds a new Node, with a value, to the back of the queue.
- T dequeue ()

  Deletes the Node at the front of the queue.
- T peek () const

  Returns the value of the Node at the front, or head, of the queue.
- void display () const

  Prints all Nodes in the queue, highlighting the value at the front and back of the queue.

7.2.1 Detailed Description

template<typename T>
class Queue< T >

Definition at line 17 of file queue.h.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Queue()

```
template<typename T>
Queue< T >::Queue ()
```

Initializes an empty queue.

Head and tail are set to nullptr, allowing the queue to be empty.

Definition at line 21 of file queue.cpp.

```
00022 {
00023     head = tail = nullptr;
00024 }
```

7.2.2.2 ~Queue()

```
template<typename T>
Queue< T >::~Queue ()
```

Clears the queue and releases all dynamically allocated resources.

Deletes all the Nodes stored in the queue, thus releasing all allocated memory.

Definition at line 35 of file queue.cpp.

```
00036 {
00037     Node<T>* tmp = head;
00038
00039     while (tmp != nullptr)
00040     {
00041         Node<T>* deleteNode = tmp;
00042         tmp = tmp->getNext();
00043         delete deleteNode;
00044     }
00045 }
```

7.2.3 Member Function Documentation

7.2.3.1 dequeue()

```
template<typename T>
T Queue< T >::dequeue ()
```

Deletes the Node at the front of the queue.

If the queue is empty, then an error message is thrown. Otherwise, a new pointer "deleteNode" that references to the front of the queue (pointer "head") is initialized, while another variable stores the value stored in "head" to return it.

The front of the queue becomes the Node after the previous head. Finally, "deleteNode" is deleted.

Returns

The value stored in the deleted Node.

< If "head" is nullptr, so will "tail" be, meaning the queue is now empty.

Definition at line 123 of file queue.cpp.

```
{
    if (emptyQueue())
    {
        throw runtime_error("The queue has no elements yet!");
    }
    else
    {
        Node<T>* deleteNode = head;
        T nodeValue = head->getValue();

        head = head->getNext();
        if (head == nullptr)
        {
            tail = nullptr;
        }

        delete deleteNode;
        return nodeValue;
    }
}
```

7.2.3.2 display()

```
template<typename T>
void Queue< T >::display () const
```

Prints all Nodes in the queue, highlighting the value at the front and back of the queue.

If the queue is empty, then an error message is thrown. Otherwise, a new pointer "tmp" that references to the front of the queue (pointer "head") is initialized. By using a while loop, the function prints all the Nodes in the queue, then provides the value of the Nodes stored at the front and back of the list.

Definition at line 182 of file queue.cpp.

```
{
    if (emptyQueue())
    {
        cout « "The queue has no elements yet!";
    }
    else
    {
        Node<T>* tmp = head;

        while (tmp != nullptr)
        {
            cout « tmp->getValue() « "|";
            tmp = tmp->getNext();
        }

        cout « "\nThe number of Nodes in the queue is: " « getSize();
        cout « "\n--> First value of the queue (head): " « head->getValue();
        cout « "\n--> Last value of the queue (tail): " « tail->getValue();
    }
}
```

7.2.3.3 emptyQueue()

```
template<typename T>
bool Queue< T >::emptyQueue () const
```

Returns true if the queue has no Nodes, and false otherwise.

Returns

True if the queue has no Nodes. False if it has at least one Node.

< If "head" is nullptr, so will "tail" be.

Definition at line 55 of file queue.cpp.

```
00056 {
00057     return head == nullptr;
00058 }
```

7.2.3.4 enqueue()

```
template<typename T>
void Queue< T >::enqueue (
            Node< T > * newNode)
```

Adds a new Node, with a value, to the back of the queue.

If the queue is empty, the new Node becomes both the head and tail of the stack. Otherwise, the pointer of the new Node is set to the memory address of "head", then the new Node becomes the back of the queue.

Parameters

newNode	Memory address of the new Node.

Definition at line 95 of file queue.cpp.

```
00096 {
00097     if (emptyQueue())
00098     {
00099         head = tail = newNode;
00100     }
00101     else
00102     {
00103         tail->insertNext(newNode);
00104         tail = newNode;
00105     }
00106 }
```

7.2.3.5 getSize()

```
template<typename T>
int Queue< T >::getSize () const
```

Returns the number of Nodes in the queue by traversing the list.

Returns

The number of Nodes in the queue.

Definition at line 68 of file queue.cpp.

```
00069 {
00070     int count = 0;
00071     Node<T>* tmp = head;
00072
00073     while (tmp != nullptr)
00074     {
00075         count++;
00076         tmp = tmp->getNext();
00077     }
00078
00079     return count;
00080 }
```

7.2.3.6 peek()

```
template<typename T>
T Queue< T >::peek () const
```

Returns the value of the Node at the front, or head, of the queue.

If the queue is empty, then an error message is thrown. Otherwise, it returns the value inside "head".

Returns

The value stored in the front of the stack.

Definition at line 157 of file queue.cpp.

```
{
    if (emptyQueue())
    {
        throw runtime_error("The queue has no elements yet!");
    }
    else
    {
        return head->getValue();
    }
}
```

The documentation for this class was generated from the following files:

- headers/queue.h
- src/queue.cpp

Chapter 8

File Documentation

8.1 headers/node.h File Reference

Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".

```
#include <string>
#include "../src/node.cpp"
```

Classes

- class Node< T >

8.1.1 Detailed Description

Defines the structure of a Node. Its structure consists of a type T "value", and a pointer "next".

A node can store a value of type T in "value", whereas the pointer "next" that references to the next Node in the list. This file defines the constructors of the functions of a Node.

Definition in file node.h.

8.2 node.h

Go to the documentation of this file.

```
#pragma once
#include <string>


template <typename T>
class Node
{
    private:
        T value;
        Node<T>* next;

    public:
        Node(T data, Node<T>* nextNode);
        Node(T data);
        Node();

        void insertValue(T data);
        void insertNext(Node<T>* nextNode);
        T getValue() const;
        Node<T>* getNext() const;
};

#include "../src/node.cpp"
```

8.3 headers/queue.h File Reference

Defines the structure of the Queue. Its structure consists of two pointers called "head" and "tail", respectively.

```
#include <string>
#include "node.h"
#include "../src/queue.cpp"
```

Classes

- class Queue< T >

8.3.1 Detailed Description

Defines the structure of the Queue. Its structure consists of two pointers called "head" and "tail", respectively.

A queue is a linear data structure that follows the FIFO (First-in, First-out) method, thus making the core of its functions the first (front) and last (back) elements, or in this case Nodes, added to the queue. This file defines the constructors of the functions of the queue.

Definition in file queue.h.

8.4 queue.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include <string>
00003 #include "node.h"
00004
00015
00016 template <typename T>
00017 class Queue
00018 {
00019     private:
00020
00021         Node<T>* head;
00022         Node<T>* tail;
00023
00024     public:
00025
00026         Queue();
00027         ~Queue();
00028
00029         bool emptyQueue() const;
00030         int getSize() const;
00031
00032         void enqueue(Node<T>* data);
00033         T dequeue();
00034
00035         T peek() const;
00036         void display() const;
00037
00038 };
00039
00040 #include "../src/queue.cpp"
```

8.5 README.md File Reference

8.6 src/main.cpp File Reference

The complete program of the dynamic queue. Includes a menu function so that the user may input any desired value.

```
#include <iostream>
#include "node.h"
#include "queue.h"
```

Functions

- int menu ()

  Menu that manages user input.
- int main ()

8.6.1 Detailed Description

The complete program of the dynamic queue. Includes a menu function so that the user may input any desired value.

The options are as followed:

- Option 1 allows the user to input any value of type int.
- Option 2 allows the user to delete the Node at the front of the queue.
- Option 3 allows the user to view the value at the front of the queue.
- Option 5 allows the user to view all the Nodes of the queue, as well as the first and last elements.
- Option 5 allows the user to exit the program.

Definition in file main.cpp.

8.6.2 Function Documentation

8.6.2.1 main()

```
int main ()
```

< Initializes a stack of type int.

< Attempts to delete the front of the queue.

< Sends a message if an error occurs.

< Attempts to retrieve the value in front of the queue.

< Sends a message if an error occurs.

Definition at line 21 of file main.cpp.

```
{
    Queue<int> queue;

    while (true)
    {

        switch (menu())
        {
        case 1:
        {
            int value;
            cout « "\nWhat value is to be stored in the new Node?: ";
            cin » value;

            queue.enqueue(new Node<int>(value));
            cout « "\nThe node has been added with the value of " « value « "!";
            break;
        }
        case 2:
        {
            try
            {
                cout « "\nThe value stored in the front of the queue is: " « queue.dequeue();
                cout « "\nThe front of the queue has been deleted!";
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 3:
        {
            try
            {
                cout « "\nThe value stored in the front of the queue is: " « queue.peek();
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 4:
        {
            queue.display();
            break;
        }
        case 5:
        {
            cout « "\nExiting the program...";
            exit(0);
            break;
        }
        }
    }
}
```

8.6.2.2 menu()

```
int menu ()
```

Menu that manages user input.

Returns an option to the main function.

Definition at line 90 of file main.cpp.

```
{
    int option;
    do
    {
        cout « "\n\n\n\t--- MENU! ---";
        cout « "\n1) Push.\n";
```

```
        cout « "2) Pop.\n";
        cout « "3) Peek.\n";
        cout « "4) Display.\n";
        cout « "5) Exit.\n";
        cout « "\n--> Choose an option: ";
        cin » option;
    } while (option < 1 || option > 5);
    return option;
}
```

8.7 main.cpp

Go to the documentation of this file.

```
#include <iostream>
#include "node.h"
#include "queue.h"


int menu();
using namespace std;

int main()
{
    Queue<int> queue;

    while (true)
    {

        switch (menu())
        {
        case 1:
        {
            int value;
            cout « "\nWhat value is to be stored in the new Node?: ";
            cin » value;

            queue.enqueue(new Node<int>(value));
            cout « "\nThe node has been added with the value of " « value « "!";
            break;
        }
        case 2:
        {
            try
            {
                cout « "\nThe value stored in the front of the queue is: " « queue.dequeue();
                cout « "\nThe front of the queue has been deleted!";
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 3:
        {
            try
            {
                cout « "\nThe value stored in the front of the queue is: " « queue.peek();
            }
            catch(const std::exception& e)
            {
                std::cerr « e.what() « '\n';
            }

            break;
        }
        case 4:
        {
            queue.display();
            break;
        }
        case 5:
        {
            cout « "\nExiting the program...";
            exit(0);
            break;
        }
        }
    }
}
```

```

/*************************************************************************/


int menu()
{
    int option;
    do
    {
        cout « "\n\n\n\t--- MENU! ---";
        cout « "\n1) Push.\n";
        cout « "2) Pop.\n";
        cout « "3) Peek.\n";
        cout « "4) Display.\n";
        cout « "5) Exit.\n";
        cout « "\n--> Choose an option: ";
        cin » option;
    } while (option < 1 || option > 5);
    return option;
}

/*************************************************************************/

```

8.8 src/node.cpp File Reference

Implementation of the constructors of the Node.

```
#include "node.h"
```

8.8.1 Detailed Description

Implementation of the constructors of the Node.

Definition in file node.cpp.

8.9 node.cpp

Go to the documentation of this file.

```
#include "node.h"


/*************************************************************************/


template <typename T>
Node<T>::Node(T data, Node<T>* nextNode)
{
    value = data;
    next = nextNode;
}

/*************************************************************************/


template <typename T>
Node<T>::Node(T data)
{
    value = data;
    next = nullptr;
}

/*************************************************************************/


template <typename T>
```

```
Node<T>::Node()
{
    value = T();
    next = nullptr;
}

/*********************************************************************/


template <typename T>
void Node<T>::insertValue(T data)
{
    value = data;
}

/*********************************************************************/


template <typename T>
void Node<T>::insertNext(Node<T>* nextNode)
{
    next = nextNode;
}

/*********************************************************************/


template <typename T>
T Node<T>::getValue() const
{
    return value;
}

/*********************************************************************/


template <typename T>
Node<T>* Node<T>::getNext() const
{
    return next;
}
```

8.10 src/queue.cpp File Reference

Implementation of the constructors of the queue.

```
#include <iostream>
#include "queue.h"
#include "node.h"
```

8.10.1 Detailed Description

Implementation of the constructors of the queue.

Definition in file queue.cpp.

8.11 queue.cpp

Go to the documentation of this file.

```
#include <iostream>
#include "queue.h"
#include "node.h"


using namespace std;
```

```

/*****************************************************************************/


template <typename T>
Queue<T>::Queue()
{
    head = tail = nullptr;
}

/*****************************************************************************/


template <typename T>
Queue<T>::~Queue()
{
    Node<T>* tmp = head;

    while (tmp != nullptr)
    {
        Node<T>* deleteNode = tmp;
        tmp = tmp->getNext();
        delete deleteNode;
    }
}

/*****************************************************************************/


template <typename T>
bool Queue<T>::emptyQueue() const
{
    return head == nullptr;
}

/*****************************************************************************/


template <typename T>
int Queue<T>::getSize() const
{
    int count = 0;
    Node<T>* tmp = head;

    while (tmp != nullptr)
    {
        count++;
        tmp = tmp->getNext();
    }

    return count;
}

/*****************************************************************************/


template <typename T>
void Queue<T>::enqueue(Node<T>* newNode)
{
    if (emptyQueue())
    {
        head = tail = newNode;
    }
    else
    {
        tail->insertNext(newNode);
        tail = newNode;
    }
}

/*****************************************************************************/


template <typename T>
T Queue<T>::dequeue()
{
    if (emptyQueue())
    {
        throw runtime_error("The queue has no elements yet!");
    }
    else
    {
        Node<T>* deleteNode = head;
        T nodeValue = head->getValue();

        head = head->getNext();
        if (head == nullptr)
```

```
        {
            tail = nullptr;
        }

        delete deleteNode;
        return nodeValue;
    }
}

/*************************************************************************/


template <typename T>
T Queue<T>::peek() const
{
    if (emptyQueue())
    {
        throw runtime_error("The queue has no elements yet!");
    }
    else
    {
        return head->getValue();
    }
}


/*************************************************************************/


template <typename T>
void Queue<T>::display() const
{
    if (emptyQueue())
    {
        cout « "The queue has no elements yet!";
    }
    else
    {
        Node<T>* tmp = head;

        while (tmp != nullptr)
        {
            cout « tmp->getValue() « "|";
            tmp = tmp->getNext();
        }

        cout « "\nThe number of Nodes in the queue is: " « getSize();
        cout « "\n--> First value of the queue (head): " « head->getValue();
        cout « "\n--> Last value of the queue (tail): " « tail->getValue();
    }
}
```

Index

~Queue
    Queue< T >, 17

dequeue
    Queue< T >, 17
display
    Queue< T >, 18
Dynamic Queue in C++, 1

emptyQueue
    Queue< T >, 18
enqueue
    Queue< T >, 19

getNext
    Node< T >, 15
getSize
    Queue< T >, 19
getValue
    Node< T >, 15

headers Directory Reference, 11
headers/node.h, 21
headers/queue.h, 22

insertNext
    Node< T >, 15
insertValue
    Node< T >, 15

main
    main.cpp, 23
main.cpp
    main, 23
    menu, 24
menu
    main.cpp, 24

Node
    Node< T >, 14
Node< T >, 13
    getNext, 15
    getValue, 15
    insertNext, 15
    insertValue, 15
    Node, 14

peek
    Queue< T >, 19

Queue
    Queue< T >, 17
Queue< T >, 16
    ~Queue, 17
    dequeue, 17
    display, 18
    emptyQueue, 18
    enqueue, 19
    getSize, 19
    peek, 19
    Queue, 17

README.md, 23

src Directory Reference, 11
src/main.cpp, 23, 25
src/node.cpp, 26
src/queue.cpp, 27

Tests, 3